Conjugate Gradient Methods for Optimization

Chapter 7 of Numerical Analysis

Yiping Cheng
ypcheng@bjtu.edu.cn
27-Sep-2022

School of Electronic and Information Engineering
Beijing Jiaotong University

Chapter Contents

1 Introduction to Optimization
- Overview of Optimization Problems
- Fundamentals of Unconstrained Continuous Local Optimization
- Overview of Optimization Algorithms

2 The Linear Conjugate Gradient Method
- Algorithm Description
- Algorithm Properties

3 Nonlinear Conjugate Gradient Methods
- Description of the Methods
- Elements of Line Search

Introduction to Optimization

Mathematical formulation of optimization

Mathematically speaking, optimization is the minimization of a function subject to constraints on its variables. We use the following notation:

– x is the vector of variables, also called unknowns or parameters;

– f is the objective function, a scalar function of x that we want to minimize;

– c_i are constraint functions, which are scalar functions of x that define certain equations and inequalities that the unknown vector x must satisfy.

Using this notation, the optimization problem can be written as follows:

$$\min_{x \in \mathbb{R}^n} f(x) \text{ subject to } c_i(x) = 0, i \in \boldsymbol{E}, \text{ and } c_i(x) \geq 0, i \in \boldsymbol{I} \tag{1}$$

Here $\boldsymbol{E}$ and $\boldsymbol{I}$ are sets of indices for equality and inequality constraints, respectively.

Unconstrained versus constrained optimization

An important distinction is between optimization problems that have constraints on the variables and those that do not.

Unconstrained optimization problems, for which we have $\boldsymbol{E} = \boldsymbol{I} = \varnothing$ in (1), arise directly in many practical applications. Even for some problems with natural constraints on the variables, it may be safe to disregard them as they do not affect on the solution and do not interfere with algorithms.

Constrained optimization problems arise from models in which constraints play an essential role, for example in imposing budgetary constraints in an economic problem or shape constraints in a design problem. These constraints may be simple bounds such as $0 \leq x_1 \leq 100$, more general linear constraints such as $\sum_i x_i \leq 1$, or nonlinear inequalities that represent complex relationships among the variables.

In this chapter we study only unconstrained optimization problems.

Continuous versus discrete optimization

The defining feature of a **discrete optimization** problem is that the unknown x is drawn from a finite (but often very large) set. By contrast, the feasible set for **continuous optimization** problems is usually uncountably infinite, as when the components of x are allowed to be real numbers. Continuous optimization problems are normally easier to solve because the smoothness of the functions makes it possible to use objective and constraint information at a particular point x to deduce information about the function's behavior at all points close to x. In discrete problems, by contrast, the behavior of the objective and constraints may change significantly as we move from one feasible point to another, even if the two points are "close" by some measure.

In this chapter we study only continuous optimization problems.

Local versus global optimization

Local optimization algorithms seek only a local solution, a point at which the objective function is smaller than at all other feasible nearby points. They do not always find the global solution, which is the point with lowest function value among all feasible points. **Global optimization** algorithms seek to find the global solution.

Global solutions are needed in some applications, but for many problems they are difficult to recognize and even more difficult to locate. For many problems, local solutions are also global solutions. General nonlinear problems, both constrained and unconstrained, may possess local solutions that are not global solutions.

In this chapter we study only local optimization algorithms.

Formulation of unconstrained continuous optimization

In unconstrained continuous optimization, we minimize an objective function that depends on real variables, with no restrictions at all on the values of these variables. The mathematical formulation is

$$\min_{x \in \mathbb{R}^n} f(x) \tag{2}$$

where $f : \mathbb{R}^n \to \mathbb{R}$ is a smooth function.

What is a solution?

Generally, we would be happiest if we found a **global minimizer** of f, a point where the function attains its least value. A formal definition is

$$\text{A point } x^* \text{ is a global minimizer of } f \text{ if } f(x^*) \leq f(x) \text{ for all } x \in \mathbb{R}^n.$$

The global minimizer can be difficult to find. Most algorithms are able to find only a **local minimizer**, which is a point that achieves the smallest value of f in its neighborhood. Formally, we say:

$$\text{A point } x^* \text{ is a local minimizer of } f \text{ if there is a neighborhood } \mathcal{N} \text{ of } x^*$$

$$\text{such that } f(x^*) \leq f(x) \text{ for all } x \in \mathcal{N}.$$

A point that satisfies this definition is sometimes called a **weak local minimizer**.

This terminology distinguishes it from a **strict local minimizer**, which is the outright winner in its neighborhood. Formally,

A point x^* is a strict local minimizer of f if there is a neighborhood $\mathcal{N}$ of x^*

such that $f(x^*) < f(x)$ for all $x \in \mathcal{N}$ with $x \neq x^*$.

Recognizing a local minimum

From the definitions given above, it might seem that the only way to find out whether a point x^* is a local minimizer is to examine all the points in its immediate vicinity, to make sure that none of them has a smaller function value. When the function f is smooth, however,there are more efficient and practical ways to identify local minimum. In particular, if f is twice continuously differentiable, we may be able to tell that x^* is a local minimizer (and possibly a strict local minimizer) by examining just the gradient $\nabla f(x^*)$ and the Hessian $\nabla^2 f(x^*)$.

The mathematical tool used to study minimizers of smooth functions is Taylor's theorem. Because this theorem is central to our analysis throughout the chapter, we state it now. Its proof can be found in any calculus textbook.

Theorem (Taylor's Theorem)

Suppose that $f : \mathbb{R}^n \to \mathbb{R}$ is continuously differentiable and that $p \in \mathbb{R}^n$. Then we have that

$$f(x+p) = f(x) + \nabla f(x+tp)^T p \tag{3}$$

for some $t \in (0,1)$.
Moreover, if f is twice continuously differentiable, we have that

$$f(x+p) = f(x) + \nabla f(x)^T p + \frac{1}{2} p^T \nabla^2 f(x+tp)\, p \tag{4}$$

for some $t \in (0,1)$.

Necessary conditions for optimality are derived by assuming that x^* is a local minimizer and then proving facts about $\nabla f(x^*)$ and $\nabla^2 f(x^*)$.

Theorem (First-Order Necessary Condition)

If x^* is a local minimizer of f and f is continuously differentiable in an open neighborhood of x^*, then $\nabla f(x^*) = 0$.

We call x^* a stationary point if $\nabla f(x^*) = 0$. According to this theorem, any local minimizer must be a stationary point.

Proof.

Suppose for contradiction that $\nabla f(x^*) \neq 0$. Define the vector $p = -\nabla f(x^*)$ and note that $p^T \nabla f(x^*) = -\|\nabla f(x^*)\|^2 < 0$. Because $\nabla f(x^*)$ is continuous near x^*, there is a scalar $T > 0$ such that

$$p^T \nabla f(x^* + tp) < 0, \quad \text{for all } t \in [0, T].$$

For any $\tau \in (0, T]$, we have by Taylor's theorem that

$$f(x^* + \tau p) = f(x^*) + \tau p^T \nabla f(x^* + tp), \quad \text{for some } t \in (0, \tau).$$

Therefore, $f(x^* + \tau p) < f(x^*)$ for all $\tau \in (0, T]$. We have found a direction leading away from x^* along which f decreases, so x^* is not a local minimizer, and we have a contradiction. □

Theorem (Second-Order Necessary Condition)

If x^* is a local minimizer of f and $\nabla^2 f$ exists and is continuous in an open neighborhood of x^*, then $\nabla f(x^*) = 0$ and $\nabla^2 f(x^*)$ is positive semidefinite.

Proof.

We know from the previous theorem that $\nabla f(x^*) = 0$. For contradiction, assume that $\nabla^2 f(x^*)$ is not positive semidefinite. Then we can choose a vector p such that $p^T \nabla f(x^*) p < 0$, and because $\nabla^2 f$ is continuous near x^*, there is a scalar $T > 0$ such that $p^T \nabla^2 f(x^* + tp)p < 0$ for all $t \in [0, T]$. By doing a Taylor series expansion around x^*, we have for all $\tau \in (0, T]$ and some $t \in (0, \tau)$ that

$$f(x^* + \tau p) = f(x^*) + \tau p^T \nabla f(x^*) + \frac{1}{2}\tau^2 p^T \nabla^2 f(x^* + tp)p < f(x^*).$$

As in the previous theorem, we have found a direction from x^* along which f is decreasing, and so again, x^* is not a local minimizer. □

We now describe sufficient conditions, which are conditions on the derivatives of f at the point x^* that guarantee that x^* is a local minimizer.

Theorem (Second-Order Sufficient Condition)

Suppose that $\nabla^2 f$ is continuous in an open neighborhood of x^* and that $\nabla f(x^*) = 0$ and $\nabla^2 f(x^*)$ is positive definite. Then x^* is a strict local minimizer of f.

Note that this second-order sufficient condition guarantees something stronger than the necessary conditions discussed earlier; namely, that the minimizer is a strict local minimizer. Note too that the second-order sufficient conditions are not necessary: A point x^* may be a strict local minimizer, and yet may fail to satisfy the sufficient condition. A simple example is given by the function $f(x) = x^4$, for which the point $x^* = 0$ is a strict local minimizer at which the Hessian matrix vanishes (and is therefore not positive definite).

Proof.

Because the Hessian is continuous and positive definite at x^*, we can choose a radius $r > 0$ so that $\nabla^2 f(x)$ remains positive definite for all x in the open ball $\mathscr{D} = \{z : \|z - x^*\| < r\}$. Taking any nonzero vector p with $\|p\| < r$, we have $x^* + p \in \mathscr{D}$ and so

$$\begin{aligned} f(x^* + p) &= f(x^*) + p^T \nabla f(x^*) + \frac{1}{2} p^T \nabla^2 f(z) p \\ &= f(x^*) + \frac{1}{2} p^T \nabla^2 f(z) p, \end{aligned}$$

where $z = x^* + tp$ for some $t \in (0, 1)$. Since $z \in \mathscr{D}$, we have $p^T \nabla^2 f(z) p > 0$, and therefore $f(x^* + p) > f(x^*)$, giving the result. □

Two strategies: line search and trust region

All algorithms for unconstrained minimization require the user to supply a starting point, which we usually denote by x_0. The user with knowledge about the application and the data set may be in a good position to choose x_0 to be a reasonable estimate of the solution. Otherwise, the starting point must be chosen by the algorithm, either by a systematic approach or in some arbitrary manner.

Beginning at x_0, optimization algorithms generate a sequence of iterates $\{x_k\}_{k=0}^{\infty}$ that terminate when either no more progress can be made or when it seems that a solution point has been approximated with sufficient accuracy. In deciding how to move from one iterate x_k to the next, the algorithms use information about the function f at x_k, and possibly also information from earlier iterates x_0, x_1,..., x_{k-1}. They use this information to find a new iterate x_{k+1} with a lower function value than x_k.

There are two fundamental strategies for moving from the current point x_k to a new iterate x_{k+1}. Most of the algorithms described in this book follow one of these approaches. They are called **line search** and **trust region**, respectively.

The line search strategy

In the line search strategy, the algorithm chooses a direction p_k and searches along this direction from the current iterate x_k for a new iterate with a lower function value. The distance to move along p_k can be found by approximately solving the following one dimensional minimization problem to find a step length α:

$$\min_{\alpha>0} f(x_k + \alpha p_k). \tag{5}$$

By solving (5) exactly, we would derive the maximum benefit from the direction p_k, but an exact minimization may be expensive and is usually unnecessary. Instead, the line search algorithm generates a limited number of trial step lengths until it finds one that loosely approximates the minimum of (5). At the new point, a new search direction and step length are computed, and the process is repeated.

Optimization methods using line search

Optimization methods using line search include but are not restricted to the following:

1. **Gradient Descent** This method updates the current point by moving in the direction of the negative gradient of the objective function. A line search is performed to determine the optimal step size along this direction. It is widely used in machine learning and convex optimization.
2. **Newton's Method** This method uses the first and second derivatives (gradient and Hessian) to find the minimum. It performs a line search to determine the optimal step size along the direction given by the Newton step. It is useful in problems where the Hessian can be computed efficiently, such as in quadratic programming.
3. **Quasi-Newton Methods** These methods approximate the Hessian matrix and use the gradient to compute a search direction. A line search is then employed to find the appropriate step size. Example: BFGS (Broyden-Fletcher-Goldfarb-Shanno) is a popular quasi-Newton method. They are effective for large-scale optimization problems where the exact Hessian is computationally expensive to calculate.

4. **Limited-Memory BFGS (L-BFGS)** A variant of the BFGS method that stores only a limited number of vectors to approximate the Hessian, which reduces memory usage. It also uses line searches for step size selection. It is commonly used in large-scale optimization problems, such as machine learning.
5. **Nonlinear Conjugate Gradient Method** This iterative optimization method generalizes the idea of the conjugate gradient method, originally designed for linear problems, to handle nonlinear objective functions. Line searches are performed along conjugate directions to find optimal step sizes. This method is particularly useful in large-scale optimization problems where storing and inverting matrices may be impractical.

The trust region strategy

In the trust region strategy, the information gathered about f is used to construct a model function m_k whose behavior near the current point x_k is similar to that of the actual objective function f. Because the model m_k may not be a good approximation of f when x is far from x_k, we restrict the search for a minimizer of m_k to some region around x_k. In other words, we find the candidate step p by approximately solving the following subproblem:

$$\min_p m_k(x_k + p), \tag{6}$$

where $x_k + p$ lies inside the trust region. If the candidate solution does not produce a sufficient decrease in f, we conclude that the trust region is too large, and we shrink it and re-solve (6). Usually, the trust region is a ball defined by $\|p\|_2 \leq \Delta$, where the scalar $\Delta > 0$ is called the trust-region radius. Elliptical and box-shaped trust regions may also be used.

The model m_k in (6) is usually defined to be a quadratic function of the form

$$m_k(x_k + p) = f_k + p^T \nabla f_k + \frac{1}{2} p^T B_k p, \tag{7}$$

where f_k, ∇f_k, and B_k are a scalar, vector, and matrix, respectively. As the notation indicates, f_k and ∇f_k are chosen to be the function and gradient values at the point x_k, so that m_k and f are in agreement to first order at the current iterate x_k. The matrix B_k is either the Hessian $\nabla^2 f_k$ or some approximation to it.

Optimization methods using trust region

Optimization methods using trust region include but are not restricted to the following:

1. **Trust-Region Newton's Method** This method combines the Newton's method with a trust region approach. It solves a quadratic approximation of the objective function within the trust region and adjusts the trust region size based on the step's effectiveness. It is useful for non-linear optimization problems where the second derivatives (Hessian) are available or can be approximated.
2. **Dogleg Method** This method is a specific implementation of trust-region optimization. It combines the steepest descent direction and the Newton direction into a "dogleg" path that stays within the trust region. The method moves along this path to find the optimal step. It is commonly used when the Hessian is available, particularly effective for problems where the cost of computing the Hessian is not prohibitive.
3. **Subspace Trust-Region Method** This method approximates the objective function within a subspace, often using techniques like the Lanczos method or the Arnoldi method to reduce the dimensionality of the problem. It is effective for very large-scale optimization problems where a full space trust region is computationally expensive.

The Linear Conjugate Gradient Method

Some preliminary facts in matrix analysis

Lemma (Lemma 1)

Let $f(x) = b^T x$ *where* $b \in \mathbb{R}^n$. *Then* $\nabla f(x) = b$.

Lemma (Lemma 2)

Let $f(x) = x^T Ax$ *where* $A \in \mathbb{R}^{n\times n}$. *Then* $\nabla f(x) = (A + A^T)x$.

Proof of Lemma 2.

$f(x) = \sum_{i=1}^{n} \sum_{j=1}^{n} a_{ij} x_i x_j$. So for each k,

$$\frac{\partial f}{\partial x_k} = \frac{\partial}{\partial x_k}\left(\sum_{i=1}^{n}\sum_{j=1}^{n} a_{ij} x_i x_j\right) = \sum_{i=1}^{n}\sum_{j=1}^{n} a_{ij} \frac{\partial}{\partial x_k}(x_i x_j)$$

$$= \sum_{i=1}^{n} a_{ii} \frac{\partial}{\partial x_k}(x_i^2) + \sum_{1\le i\ne j=k\le n} a_{ij} \frac{\partial}{\partial x_k}(x_i x_j) + \sum_{1\le j\ne i=k\le n} a_{ij} \frac{\partial}{\partial x_k}(x_i x_j) + \sum_{1\le i\ne j, i\ne k, j\ne k\le n} a_{ij} \frac{\partial}{\partial x_k}(x_i x_j)$$

$$= 2a_{kk}x_k + \sum_{1\le i\ne k\le n} a_{ik}x_i + \sum_{1\le j\ne k\le n} a_{kj}x_j + 0 = \left(a_{kk}x_k + \sum_{1\le i\ne k\le n} a_{ik}x_i\right) + \left(a_{kk}x_k + \sum_{1\le j\ne k\le n} a_{kj}x_j\right)$$

$$= \sum_{i=1}^{n} a_{ik}x_i + \sum_{j=1}^{n} a_{kj}x_j = \sum_{j=1}^{n} (a_{jk} + a_{kj}) x_j.$$

$\square$

Problem formulation

The linear conjugate gradient method is an iterative method for solving a linear system of equations

$$Ax = b, \tag{8}$$

where A is an $n \times n$ symmetric positive definite matrix. This problem can be stated equivalently as the following minimization problem:

$$\min f(x) \overset{def}{=} \frac{1}{2} x^T A x - b^T x. \tag{9}$$

That is, both (8) and (9) have the same unique solution. This equivalence will allow us to interpret the linear conjugate gradient method either as an algorithm for solving linear systems or as a technique for minimizing convex quadratic functions. For future reference, we note that the gradient of f equals the residual of the linear system, that is,

$$\nabla f(x) = Ax - b. \tag{10}$$

The linear conjugate gradient method

Given x_0

$$g_0 = Ax_0 - b \tag{11}$$
$$p_0 = -g_0 \tag{12}$$

$k = 0$. Then loop

$$\alpha_k = -\frac{g_k^T p_k}{p_k^T A p_k} \tag{13}$$
$$x_{k+1} = x_k + \alpha_k p_k \tag{14}$$
$$g_{k+1} = \nabla f(x_{k+1}) = Ax_{k+1} - b \tag{15}$$
$$\beta_k = \frac{g_{k+1}^T A p_k}{p_k^T A p_k} \tag{16}$$
$$p_{k+1} = \beta_k p_k - g_{k+1} \tag{17}$$
$$k = k + 1 \tag{18}$$

Basic fact 1: $g_{k+1} = g_k + \alpha_k A p_k$

Proposition (Basic Fact 1)

For all $k \geq 0$, $g_{k+1} = g_k + \alpha_k A p_k$.

Proof.

$x_{k+1} = x_k + \alpha_k p_k$, then $Ax_{k+1} - b = Ax_k - b + \alpha_k A p_k$, and $g_{k+1} = g_k + \alpha_k A p_k$. □

Basic fact 2: $g_k = \beta_{k-1} p_{k-1} - p_k$

Proposition (Basic Fact 2)

For all $k \geq 1$, $g_k = \beta_{k-1} p_{k-1} - p_k$.

Proof.

Follows immediately from (17). □

The main theorem

Theorem

For all $k \geq 0$, the following result holds: for all $0 \leq i < k$,

$$g_k^T p_i = 0 \tag{19}$$

$$g_k^T g_i = 0 \tag{20}$$

$$p_k^T A p_i = 0; \tag{21}$$

and if $g_k = 0$, then $p_k = 0$, otherwise $g_k^T p_k = -g_k^T g_k < 0$.

Proof.

It is obvious that the result holds for $k = 0$.
(to be continued ...) □

Proof.

Suppose the result holds for less than or equal to k. We now show that the result continues to hold for $k+1$.
If $0 \le i < k$, then

$$\begin{aligned} g_{k+1}^T p_i &= (g_k + \alpha_k A p_k)^T p_i && \text{by basic fact 1} \\ &= g_k^T p_i + \alpha_k p_k^T A p_i \\ &= 0 && \text{by induction hypothesis.} \end{aligned}$$

In addition,

$$\begin{aligned} g_{k+1}^T p_k &= (g_k + \alpha_k A p_k)^T p_k \\ &= g_k^T p_k + \alpha_k p_k^T A p_k \\ &= 0 && \text{by definition of } \alpha_k. \end{aligned}$$

(to be continued ...) □

Proof.

$$
\begin{aligned}
g_{k+1}^T g_0 &= -g_{k+1}^T p_0 \\
&= 0 && \text{just proved.}
\end{aligned}
$$

If $1 \le i \le k$, then

$$
\begin{aligned}
g_{k+1}^T g_i &= g_{k+1}^T g_i \\
&= g_{k+1}^T (\beta_{i-1} p_{i-1} - p_i) && \text{by basic fact 2} \\
&= 0 && \text{just proved.}
\end{aligned}
$$

(to be continued ...) □

Proof.

If $0 \le i < k$, then

$$
\begin{aligned}
p_{k+1}^T A p_i &= (\beta_k p_k - g_{k+1})^T A p_i && \text{by (17)} \\
&= \beta_k p_k^T A p_i - g_{k+1}^T A p_i \\
&= -g_{k+1}^T A p_i && \text{by induction hypothesis} \\
&= \begin{cases} 0, & \text{if } p_i = 0 \\ -g_{k+1}^T \frac{g_{i+1} - g_i}{\alpha_i}, & \text{if } p_i \neq 0 \end{cases} && \text{by induction hypothesis and basic fact 1} \\
&= 0 && \text{previously proved.}
\end{aligned}
$$

In addition,

$$
\begin{aligned}
p_{k+1}^T A p_k &= (\beta_k p_k - g_{k+1})^T A p_k \\
&= \beta_k p_k^T A p_k - g_{k+1}^T A p_k \\
&= 0 && \text{by definition of } \beta_k.
\end{aligned}
$$

(to be continued ...) □

Proof.

If $g_{k+1} = 0$, then by (16), $p_k = 0$ or $\beta_k = 0$, and by (17),

$$p_{k+1} = \beta_k p_k - g_{k+1} = 0.$$

If $g_{k+1} \neq 0$, then

$$\begin{aligned} g_{k+1}^T p_{k+1} &= g_{k+1}^T(\beta_k p_k - g_{k+1}) && \text{by (17)} \\ &= \beta_k g_{k+1}^T p_k - g_{k+1}^T g_{k+1} \\ &= -g_{k+1}^T g_{k+1} && \text{previously proved} \\ &< 0. \end{aligned}$$

The proof is thus complete. □

Linear conjugate gradient method converges in at most n steps

Theorem

$g_n = Ax_n - b = 0.$

Proof.

Suppose for contradiction that $g_n \neq 0$. Then from the algorithm we see that for all $0 \leq k \leq n$, $g_k \neq 0$. Now since $g_0, \cdots, g_{n-1}, g_n$ are mutually orthogonal as we have shown, they span an $(n+1)$-dimensional vector space. But this space is contained in $\mathbb{R}^n$ which is n-dimensional, a contradiction. □

In fact, the algorithm may converge in less than n steps. That is, if x_0 is suitably chosen, it is possible that $g_m = 0$ and $Ax_m = b$ for some $m < n$. For example, if x_0 is chosen to be $A^{-1}b$, then $g_0 = 0$.

"Conjugate gradient" is a misnomer

Two vectors, p and q, are said to be conjugate with respect to A (a symmetric matrix), if $p^T Aq = 0$.

Since the gradients g_k are mutually orthogonal, the term "conjugate gradient method" is actually a misnomer. It is the search directions, not the gradients, that are conjugate with respect to A.

Equivalent formulas for α_k and β_k

From the above properties, we obtain

$$\alpha_k = -\frac{g_k^T p_k}{p_k^T A p_k} = \frac{g_k^T g_k}{p_k^T A p_k}.$$

$$\beta_k = \frac{g_{k+1}^T A p_k}{p_k^T A p_k} = \frac{g_{k+1}^T \frac{g_{k+1}-g_k}{\alpha_k}}{p_k^T A p_k} = \frac{g_{k+1}^T (g_{k+1} - g_k)}{g_k^T g_k} = \frac{g_{k+1}^T g_{k+1}}{g_k^T g_k}.$$

Linear conjugate gradient method – practical form

Given x_0

$$\begin{aligned} g_0 &= Ax_0 - b \\ p_0 &= -g_0 \end{aligned}$$

$k = 0$. While $p_k \neq 0$:

$$\begin{aligned} \alpha_k &= \frac{g_k^T g_k}{p_k^T A p_k} && (22) \\ x_{k+1} &= x_k + \alpha_k p_k \\ g_{k+1} &= \nabla f(x_{k+1}) = Ax_{k+1} - b \\ \beta_k &= \frac{g_{k+1}^T g_{k+1}}{g_k^T g_k} && (23) \\ p_{k+1} &= \beta_k p_k - g_{k+1} \\ k &= k + 1 \end{aligned}$$

Another interpretation of α_k

Up until now, we have seen the choice of α_k as solely for achieving orthogonality and conjugacy. But actually, it is equally helpful to regard α_k as the step length along the search direction that minimizes f. That is, α_k is the minimizer of the scalar function

$$\phi_k(\alpha) = f(x_k + \alpha p_k) = \frac{1}{2}(x_k + \alpha p_k)^T A(x_k + \alpha p_k) - b^T(x_k + \alpha p_k).$$

To see this, we note that

$$\phi_k(\alpha) = \frac{1}{2}p_k^T A p_k \cdot \alpha^2 + (Ax_k - b)^T p_k \cdot \alpha + \frac{1}{2}x_k^T A x_k - b^T x_k.$$

And hence it is clear that α_k is indeed the minimizer of ϕ_k.

Nonlinear Conjugate Gradient Methods

Extension of linear conjugate gradient to nonlinear functions

We have noted that the linear conjugate gradient method can be viewed as a minimization algorithm for convex quadratic functions. It is natural to ask whether we can adapt the approach to minimize general convex functions, or even general nonlinear functions. In fact, as we show in this section, nonlinear variants of the conjugate gradient are well studied and have proved to be quite successful in practice.

The Fletcher–Reeves method

Fletcher and Reeves showed how to extend the conjugate gradient method to nonlinear functions by making one simple change in the linear algorithm. In place of the formula (22) for the step length α_k (which minimizes f along the search direction p_k), we need to perform a line search that identifies an approximate minimum of the nonlinear function f along p_k. This change gives rise to the following algorithm for nonlinear optimization.

Fletcher–Reeves method – algorithm description

Given x_0

$$\begin{aligned} g_0 &= \nabla f(x_0) \\ p_0 &= -g_0 \end{aligned}$$

$k = 0$. While $p_k \neq 0$:

$$\begin{aligned} \text{compute} \quad \alpha_k & \quad \text{by line search} \\ x_{k+1} &= x_k + \alpha_k p_k \\ g_{k+1} &= \nabla f(x_{k+1}) \\ \beta_k &= \frac{g_{k+1}^T g_{k+1}}{g_k^T g_k} \\ p_{k+1} &= \beta_k p_k - g_{k+1} \\ k &= k + 1 \end{aligned}$$

Discussion about the Fletcher–Reeves method

If we choose f to be convex quadratic and α_k to be the exact minimizer, this algorithm reduces to the linear conjugate gradient method. The algorithm is appealing for large nonlinear optimization problems because each iteration requires only evaluation of the objective function and its gradient. No matrix operations are required for the step computation, and just a few vectors of storage are required.

Let us note that the orthogonality and conjugacy relationships between the gradients and directions are **lost** with general nonlinear objective functions f.

To make our algorithm complete, we need to be more precise about the choice of line search parameter α_k. From (17), we have

$$g_k^T p_k = \beta_{k-1} g_k^T p_{k-1} - g_k^T g_k. \tag{24}$$

If the line search is exact, so that α_{k-1} is a local minimizer of f along the direction p_{k-1}, we have that $g_k^T p_{k-1} = 0$. In this case we have $g_k^T p_k < 0$, so that p_k is indeed a descent direction. If the line search is not exact, however, the $\beta_{k-1} g_k^T p_{k-1}$ in (24) may dominate the other term, and we may have $g_k^T p_k > 0$, implying that p_k is actually a direction of ascent.

Fortunately, we can avoid this situation by requiring the step length α_k to satisfy the strong Wolfe conditions (which will be omitted here). We conclude that any line search procedure that yields an α_k satisfying strong Wolfe conditions will ensure that all directions p_k are descent directions for the function f.

Variants of the Fletcher–Reeves method

There are many variants of the Fletcher–Reeves method that differ from each other mainly in the choice of the parameter β_k. We now give three of them:

Polak-Ribiere formula:

$$\beta_k = \frac{g_{k+1}^T (g_{k+1} - g_k)}{g_k^T g_k}. \tag{25}$$

Hestenes-Stiefel formula:

$$\beta_k = \frac{g_{k+1}^T (g_{k+1} - g_k)}{(g_{k+1} - g_k)^T p_k}. \tag{26}$$

Dai-Yuan formula:

$$\beta_k = \frac{g_{k+1}^T g_{k+1}}{(g_{k+1} - g_k)^T p_k}. \tag{27}$$

The formulas (23,25–27) agree with one another when the function is convex quadratic.

Elements of line search

Line search is to find a suitable step size along a specified direction when minimizing a function. By choosing an appropriate step size, line search can help ensure that the algorithm converges more quickly and reliably to a local minimum.

Given the current point x_k, and the search direction p_k, where p_k must be a direction of descent (whose definition will be given later). The original purpose of line search is to find a minimizer of $\phi_k(\alpha) = f(x_k + \alpha p_k)$, where $\alpha > 0$ is a scalar representing the step size. This can involve:

- Exact line search: Finding the exact value of α that minimizes $\phi_k(\alpha)$. This approach can be computationally expensive and is often impractical.
- Inexact line search: Finding a satisfactory value of α using methods like:
  - Backtracking Line Search: Starting with a large step size and reducing it until a condition is met (often based on the Armijo condition).
  - Wolfe Conditions: These are two conditions (the Armijo condition and a curvature condition) that ensure sufficient decrease in the objective function and control the behavior of the gradient.

Search directions for line search

To ensure success of line search, the search direction must be a descent direction, or direction of descent. A descent direction is one that f is decreasing along the initial segment of the direction (but the result of the line search is not necessarily on the initial descent segment). Mathematically, this condition is equivalent to $\phi_k'(0) < 0$. Since

$$\phi_k'(0) = \frac{d f(x_k + \alpha p_k)}{d\alpha}|_{\alpha=0} = [\nabla f(x_k)]^T p_k,$$

the condition for p_k to be a descent direction is $g_k^T p_k < 0$ where $g_k = \nabla f(x_k)$.

Geometrically speaking, this condition says that the search direction must **make an acute angle** with the negative gradient direction. Often, the negative gradient direction itself is used as the search direction, which is also the direction along which f decreases fastest, or steepest. Therefore, the negative gradient direction is also called the **steepest descent** direction.